

Reframing OpenType Shaping

From serialized font tables to semantic IR, script policy, and a small runtime executor

SVG Buddy text processing architecture

A working architectural note for graphics programmers

The short version

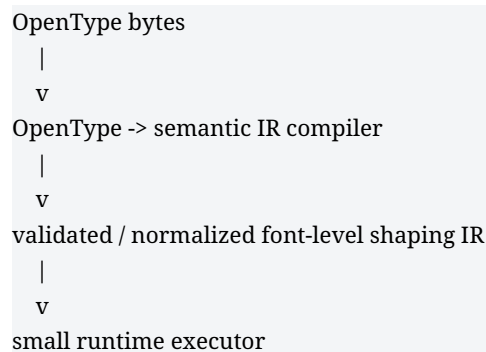
OpenType is easiest to understand when its tables are treated as compact serialized representations of semantic functions, databases, rule systems, graphs, and sometimes real instruction languages. The architecture described here compiles those serialized forms into explicit runtime semantics, while a separate Script IR expresses the writing-system policy that does not belong in the font at all.

1. Why this architecture exists

Coming to text shaping from graphics programming is initially disorienting. The surface area is dominated by four-letter table tags, Unicode property names, script-specific specifications, historical compatibility rules, and shaping engines with decades of accumulated code. It can look as though text processing is one enormous special case.

A more useful way to see the problem is that several different concerns have been serialized together and then historically interpreted together at runtime. Once those concerns are separated, the architecture becomes much easier to reason about.

For SVG Buddy, the goal is not to reproduce the structure of OpenType inside the runtime. The goal is to recover the semantic operations that OpenType encodes, compile them once, and execute those semantics through a small, well-tested engine.



A second, deliberately tiny IR handles the other half of the problem: script-specific shaping policy. That policy comes from Unicode properties and external shaping specifications, not from the GSUB/GPOS tables themselves.

The central split

Font IR answers: what can this particular font do with its glyphs? Script IR answers: what should this writing system ask the font to do, and in what order?

Why Two IRs Help in a Text Shaping Engine

Separating script knowledge from font mechanics makes the pipeline smaller, clearer, and easier to extend

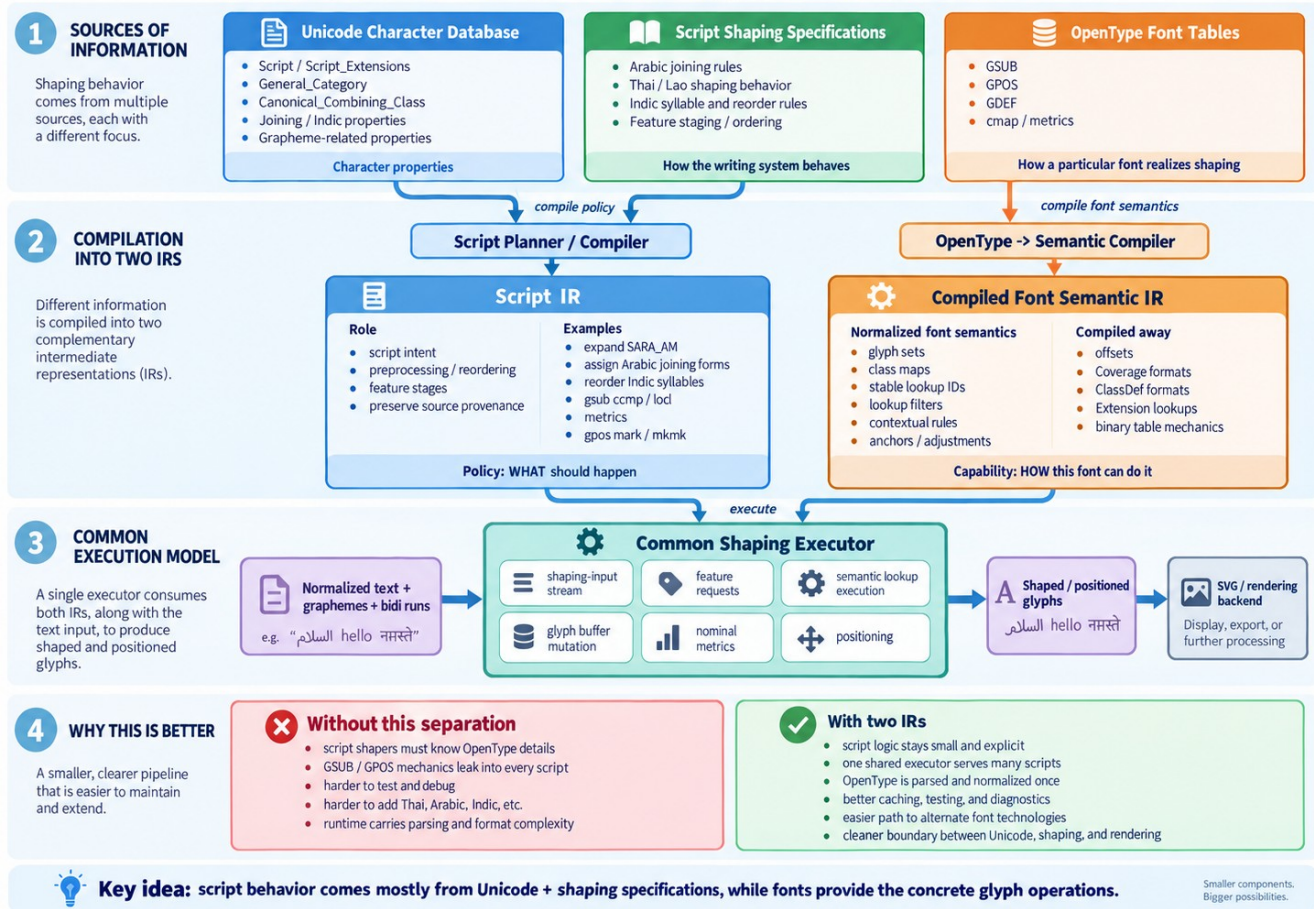


Figure 1. The two sources of shaping knowledge and the two IRs that make the separation explicit.

2. The three sources of shaping knowledge

A shaping engine does not obtain all of its behavior from the font. In practice, three information sources cooperate.

Source	What it knows	Examples
Unicode Character Database	Properties of characters	Script, Script_Extensions, General_Category, combining class, joining and Indic properties, grapheme properties
Script shaping specifications / engine policy	How a writing system behaves	Arabic joining analysis, Thai SARA AM handling, Indic syllable analysis and reordering, feature staging
OpenType font tables	How this font realizes glyph behavior	cmap mappings, GSUB substitutions, GPOS anchors and adjustments, GDEF classes, metrics

This separation is easy to miss because an OpenType shaper often exposes all three through one large code path. But they are fundamentally different kinds of knowledge.

Unicode:
properties of characters

Script specification:
rules of the writing system

Font:
concrete glyph behavior

For Arabic, for example, the shaping engine determines joining states and decides when the *isol/init/medi/fina* features should be active. The font supplies the substitutions behind those features. For Thai, the shaping engine can transform the shaping stream for SARA AM while preserving source provenance; the font supplies the glyph substitutions and mark-positioning anchors that realize the final appearance.

3. Serialized state versus realized program state

The recurring observation behind the compiler design is that an OpenType table usually contains a compact serialized representation of an implied semantic function.

The easiest example is *cmap*. Conceptually, *cmap* is just a function:

```
GlyphId glyphForCodePoint(uint32_t cp)
```

One could implement that function with a giant direct array covering the Unicode scalar range. That would be simple but wasteful. OpenType instead uses sparse and range-oriented formats. Those formats are storage choices; the semantic function remains code point to glyph ID.

The same idea scales upward. *GSUB* is not fundamentally a collection of offsets, Coverage tables and subtable format numbers. It is a serialized rule system for transforming glyph sequences. *GPOS* is a serialized rule system for geometry and attachment relationships. *GDEF* is supporting glyph semantics. *COLR v1* is a serialized paint graph. *CFF/CFF2* and TrueType hinting contain genuine instruction languages.

Compiler viewpoint

The compiler takes metadata that previously backed an implied interpreter and re-expresses it as an explicit executable API or semantic data model. Binary-format details disappear unless they still matter for execution.

Five Different Things Masquerading as 'Tables' in OpenType

A semantic taxonomy: many OpenType tables are not just tables; they encode different kinds of data models and execution behavior.

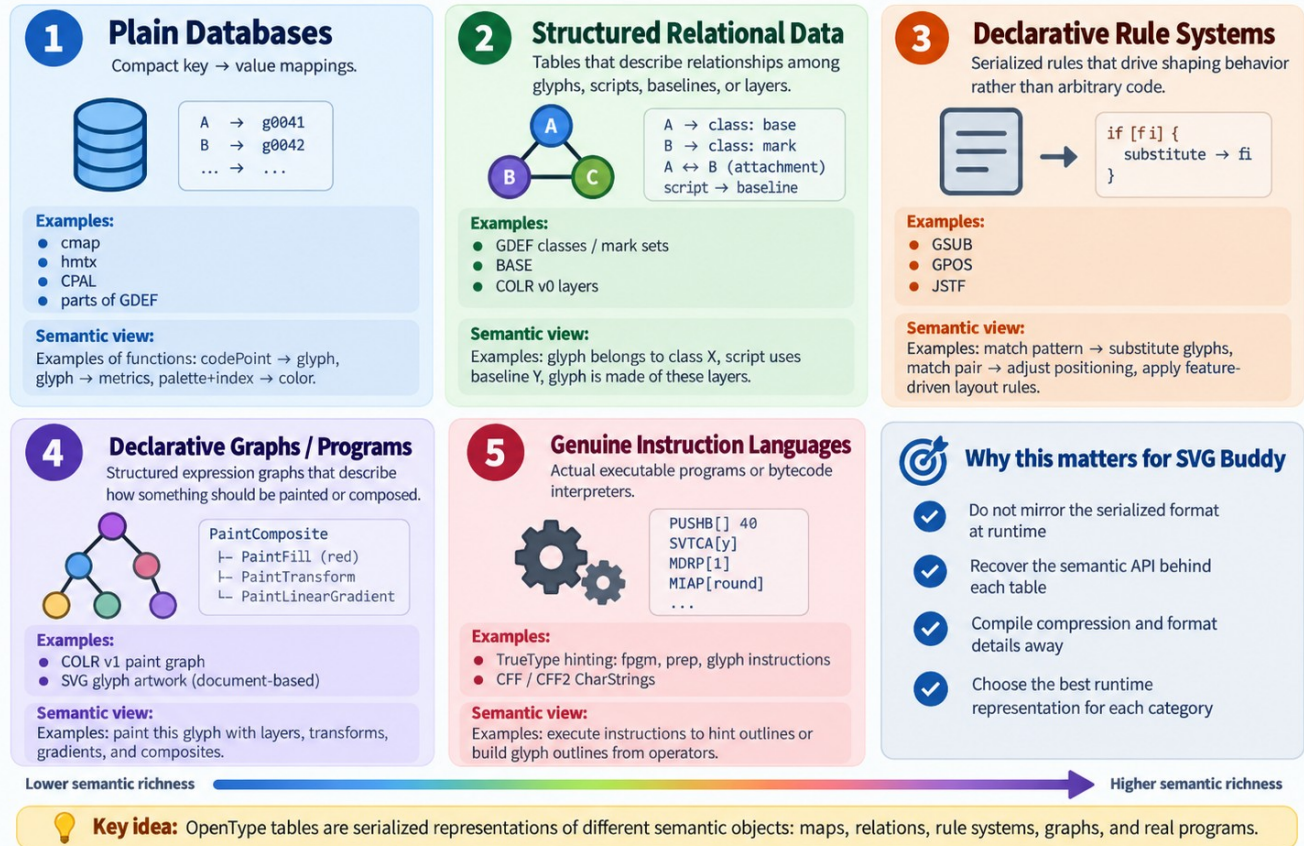


Figure 2. OpenType 'tables' are not one kind of thing. They encode several distinct semantic categories.

4. The five things masquerading as tables

Category	Semantic character	Representative tables	Natural runtime form
Plain databases	key -> value	cmap, hmtx, vmtx, CPAL	arrays, sparse maps, page tables
Structured relational data	typed relationships	GDEF, BASE, COLR v0	property maps, sets, relational structures
Declarative rule systems	match -> action	GSUB, GPOS, JSTF	semantic rules and lookup graphs
Declarative graphs/programs	expression/paint graph	COLR v1	normalized DAG / graph
Genuine instruction languages	actual executable opcodes	fpgm, prep, glyph hinting, CFF/CFF2 CharStrings	VM or compiled execution form

This taxonomy is useful because it changes the implementation question. Instead of asking how to mirror a table, ask what semantic object the table was trying to serialize in the first place.

5. OpenType tables in the text engine

The familiar four-letter names are table tags in the SFNT/OpenType directory. They are storage keys, not friendly API names. If OpenType had been designed API-first, some of them might have had names like CharacterMap, GlyphSubstitutionProgram, GlyphPositioningProgram, GlyphDefinitions, GlyphMetrics, and GlyphOutlineProvider.

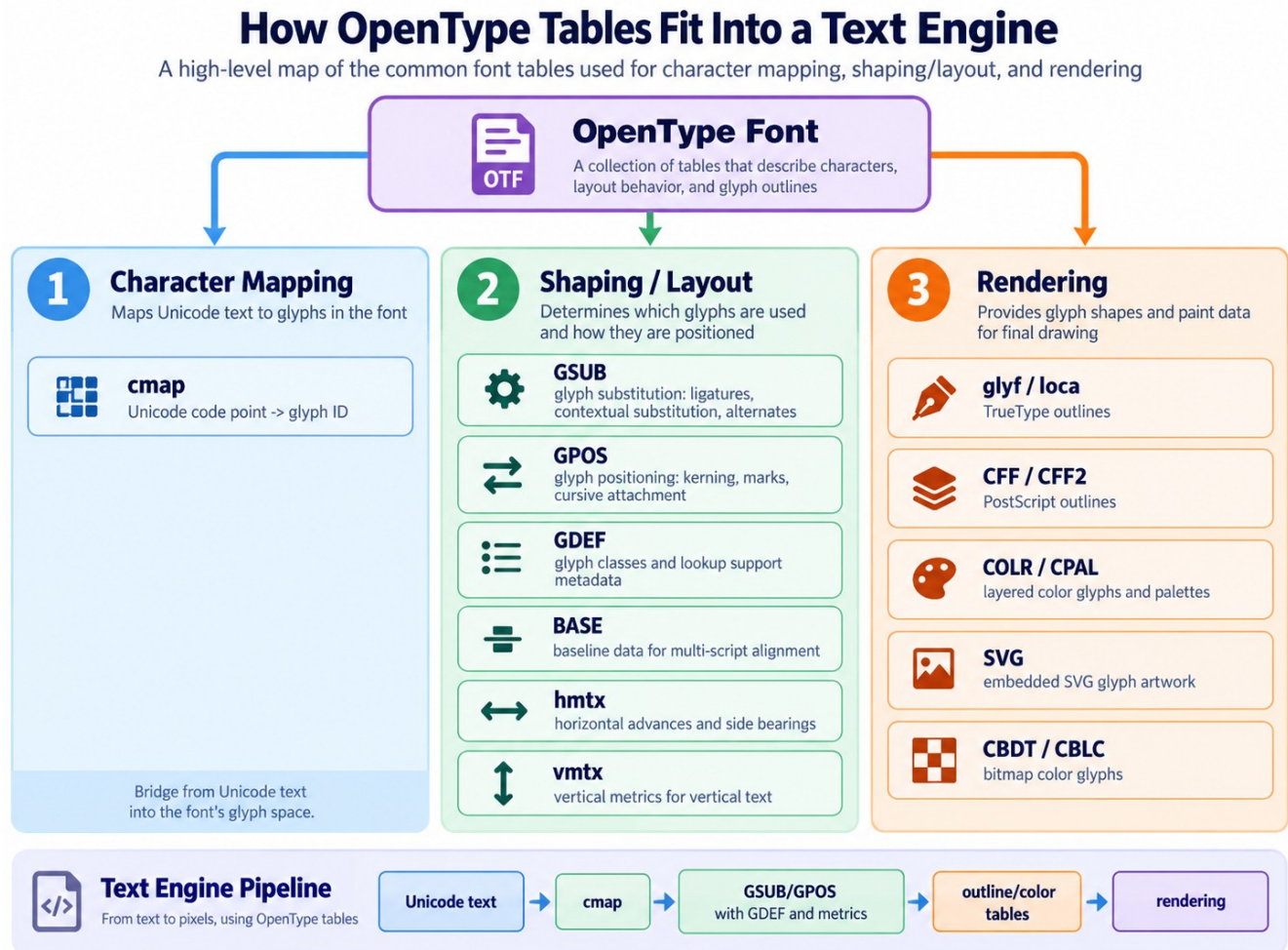


Figure 3. A practical map of common OpenType tables by character mapping, shaping/layout, and rendering.

For shaping, the core working set is relatively small:

cmap

GSUB

GDEF

GPOS

head

hhea

hmtx

+ variable-font machinery when needed

The renderer then consumes outline and paint technologies such as glyf/loca, CFF/CFF2, COLR/CPAL, SVG, or bitmap strikes.

6. The SVG Buddy text pipeline

The surrounding text pipeline remains conventional in overall shape. The architectural change is concentrated at the boundary between font data and shaping execution.

```
UTF-8
-> NFC
-> grapheme segmentation
-> Script / Script_Extensions
-> bidi
-> ShapingRunView
-> font selection/fallback
-> FontRunView
-> shaping-input preparation
-> cmap
-> GSUB
-> nominal metrics
-> GPOS
-> ShapedGlyphBuffer
-> bidi visual composition
-> absolute positioning
-> SVG glyph outline emission
```

The important point is that the pipeline does not mutate the normalized Unicode source merely because shaping needs a different internal stream.

Thai SARA AM is the motivating example:

```
source:
U+0E01 U+0E48 U+0E33

shaping stream:
U+0E01 U+0E4D U+0E48 U+0E32
```

Each shaping-input item retains source provenance, so later glyphs remain traceable to the normalized text even when the shaping stream expands, contracts, or reorders.

7. Old OpenType execution versus compiled semantic IR

The old implementation model walks OpenType structures while shaping. That means the hot path remains aware of offsets, binary formats, Coverage and ClassDef variants, Extension lookups, lookup flags, validation checks, and format-specific branching.

The new model moves those concerns to font compilation.

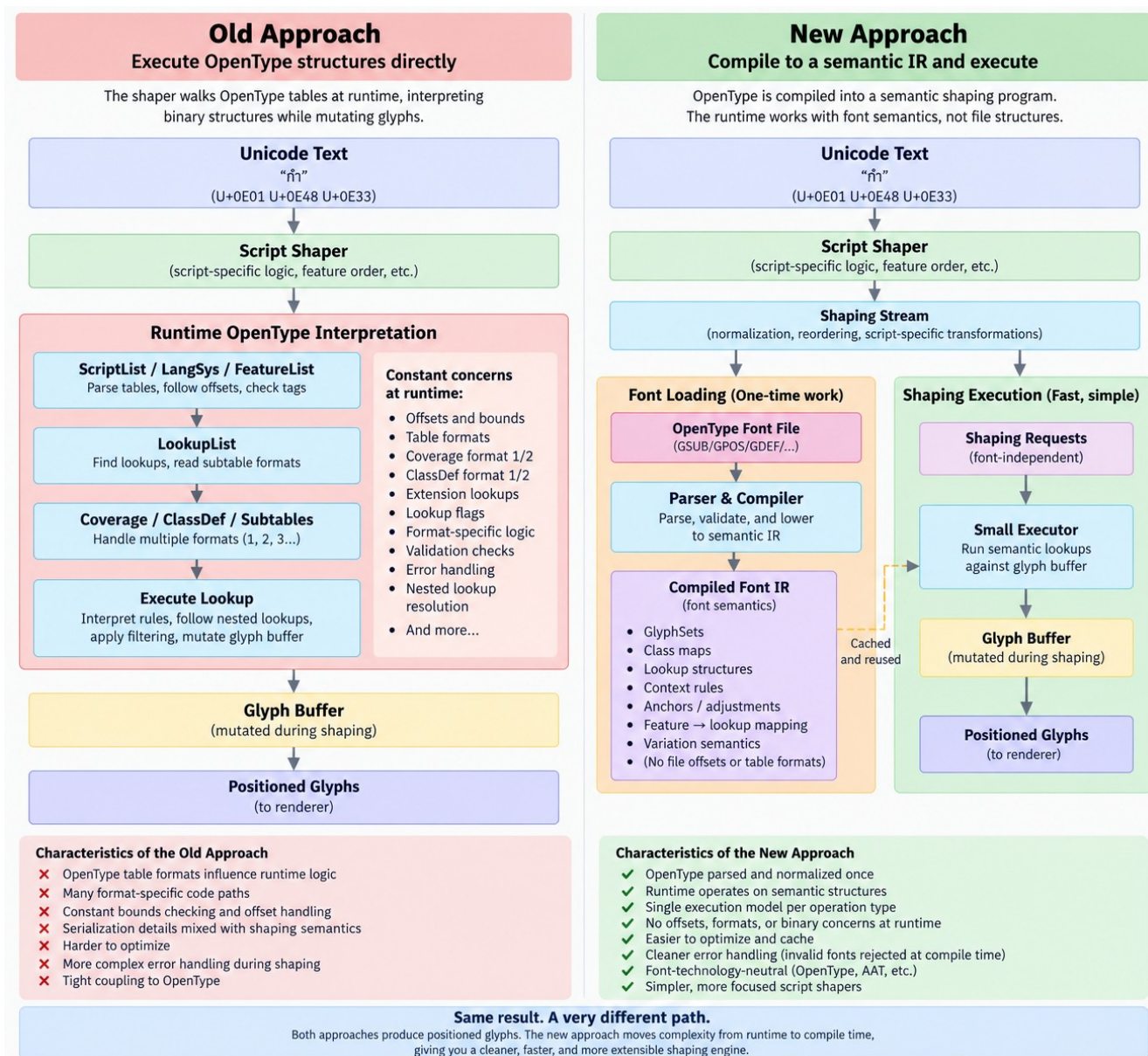


Figure 4. Runtime interpretation of OpenType structures versus one-time compilation into semantic font IR.

The result is not simply cleaner code. It changes where complexity lives.

Old runtime-oriented model	Compiled semantic model
Parse and validate while shaping	Parse and validate once
Runtime understands OpenType encodings	Runtime understands semantic operations
Coverage/ClassDef formats leak into execution	GlyphSet/GlyphClassMap abstractions
Raw LookupList indices	Stable semantic LookupId values
Extension lookups remain runtime cases	Extensions compiled away
Format-specific contextual paths	Common ContextRule / ChainContextRule forms
Malformed-data handling mixed with mutation	Compiler rejects/normalizes; executor gets trusted IR

8. The font semantic IR

GSUB is now the proof that this boundary works. All substitution lookup types have been lowered to semantic forms, including contextual and chained contextual formats, nested lookup execution, reverse-chain matching, mutable sequenceIndex behavior, recursion/operation guards, and transactional rollback.

Type 1	SingleSubst	semantic payload
Type 2	MultipleSubst	semantic payload
Type 3	AlternateSubst	semantic payload
Type 4	LigatureSubst	semantic payload
Type 5	ContextSubst	common ContextRule
Type 6	ChainContextSubst	common ChainContextRule
Type 7	ExtensionSubst	compiled away
Type 8	ReverseChainSingleSubst	semantic reverse-chain rule

The most important evidence is contextual GSUB. Formats 1, 2, and 3 are not three different shaping operations; they are three serialized encodings of contextual substitution. Once compiled, they should be executed by one semantic mechanism.

GPOS is expected to follow the same pattern, but with a different semantic vocabulary: adjustments, pair relations, anchors, mark attachments, cursive attachments, and contextual positioning.

Do not over-normalize

Semantic equivalence does not require one storage layout. PairPos Format 1 and Format 2 may share one abstract operation while still using different optimized backing structures such as a pair map and a class-by-class matrix.

9. The Script IR

The Script IR is intentionally not a general virtual machine. It is a small, declarative shaping plan that expresses script intent and phase ordering.

```
script THAI

prepare
  expand SARA_AM
  move NIKHAHIT left while THAI_ABOVE_MARK

substitute
  gsub ccmp
  gsub locl

metrics

position
  gpos kern
  gpos mark
  gpos mkmk

end
```

Arabic would use the same tiny language to express joining analysis and feature stages. Indic would use it to express syllable analysis, reordering, feature masks, and later positioning stages.

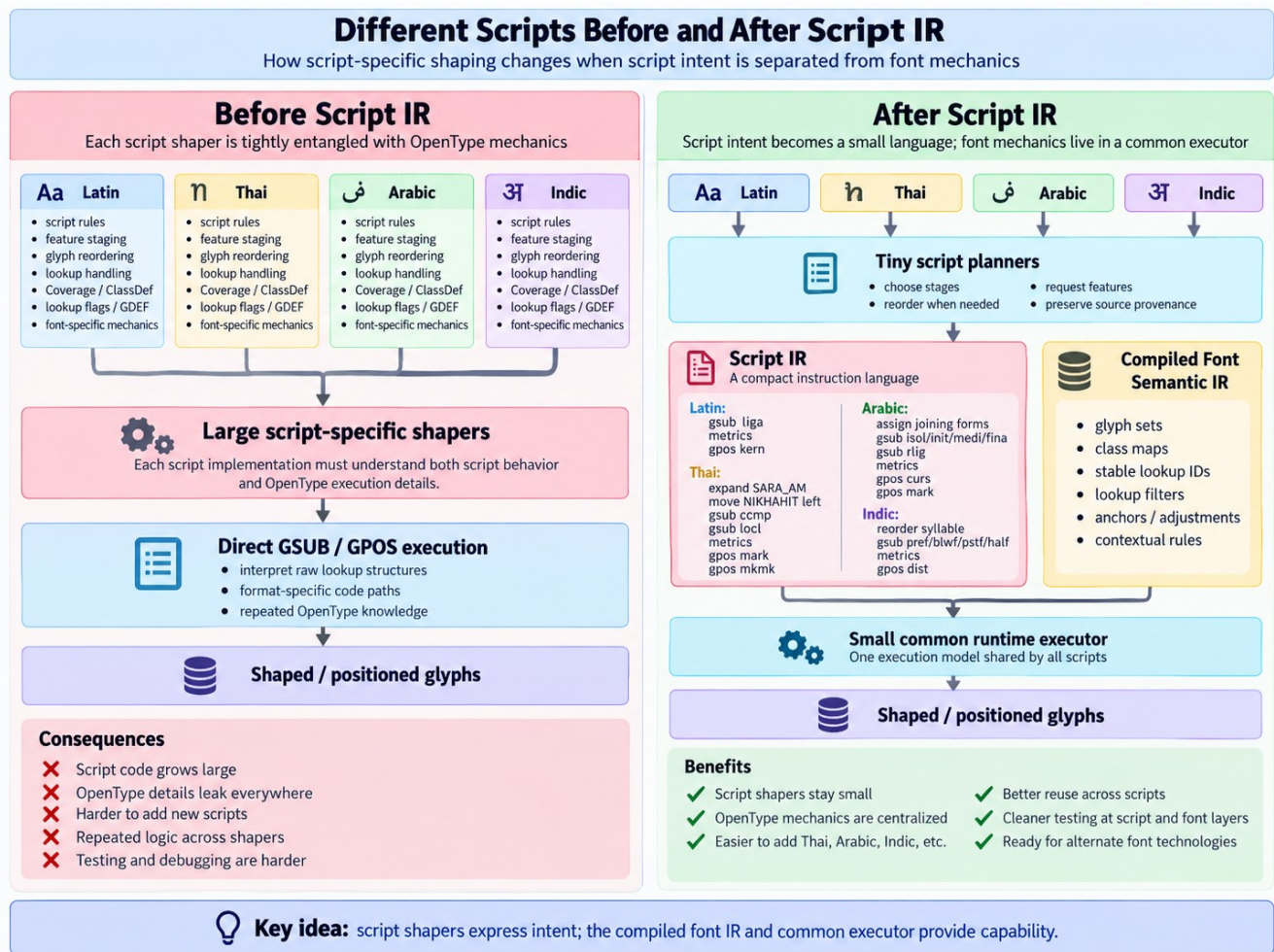


Figure 5. Script IR moves script intent out of large script-specific implementations while keeping OpenType mechanics in the common executor.

The Script IR should remain semantic rather than procedural. An operation such as `compute_joining_forms` is desirable; exposing registers, stack operations, arbitrary jumps, or character-array pointer manipulation would merely recreate another low-level VM.

The intended shape is:

```

small script program
+
small library of script-semantic operations
+
compiled font semantic IR
+
one shared glyph executor

```

10. How the two IRs meet

The two IRs are complementary rather than competing layers.

Script IR:
what should happen

Font IR:
what this font can do

Executor:
apply one to the other

A script instruction such as `gsub ccmp` should not expose raw lookup mechanics. It asks the font-side semantic layer for the lookups belonging to the `ccmp` feature in the current script/language context and executes them in the required order.

This cleanly separates writing-system policy from font mechanics. Latin can have a nearly trivial program. Thai adds preprocessing. Arabic adds joining analysis. Indic adds syllable classification and reordering. The OpenType lookup executor stays the same.

Why Two IRs Help in a Text Shaping Engine

Separating script knowledge from font mechanics makes the pipeline smaller, clearer, and easier to extend

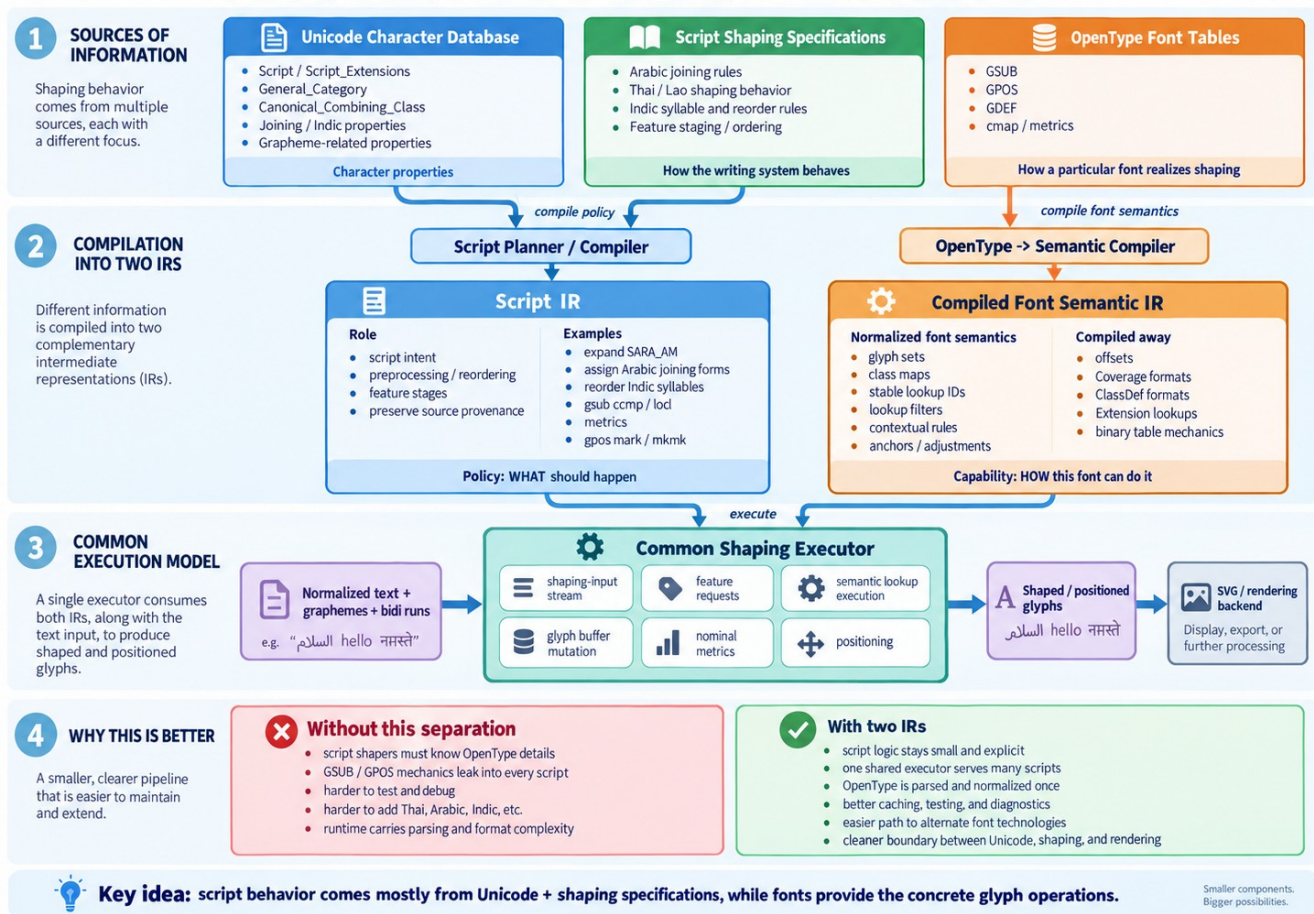


Figure 6. Unicode properties and script specifications compile toward Script IR; OpenType tables compile toward Font IR; the common executor consumes both.

11. Per-table design method

A useful design discipline is to analyze each table by the same seven questions before implementing or redesigning it.

1. What mathematical or semantic function is implied?
2. What information does the font contribute?
3. What algorithm does the engine contribute?
4. Which serialized distinctions exist mainly because of compression or storage?
5. Which distinctions have genuine semantic or performance meaning?
6. What would the API look like if OpenType did not exist?
7. What representation would I choose if I only cared about execution?

The following sheets apply that method to the tables immediately involved in shaping.

12. Design Sheet - cmap

`cmap` maps character codes, normally Unicode scalar values, to glyph IDs.

1. What mathematical/semantic function is implied?

```
GlyphId glyphForCodePoint(CodePoint cp)
```

```
GlyphId glyphForVariation(CodePoint cp, VariationSelector vs)
```

2. What information does the font contribute?

Sparse mappings from code points to glyph IDs, plus encoding/platform metadata and optional Unicode variation-sequence mappings.

3. What algorithm does the engine contribute?

Very little: select an appropriate character map, perform the lookup, and return the glyph. Most complexity is storage decoding.

4. Which serialized distinctions exist mainly because of compression or storage?

The many cmap formats largely represent alternate sparse/range/delta encodings and historical character encodings.

5. Which distinctions have genuine semantic or performance meaning?

Unicode versus legacy encodings and variation-selector semantics matter. Mapping density may influence the best runtime structure.

6. What would the API look like if OpenType did not exist?

```
glyphForCodePoint(cp)
```

```
glyphForVariation(cp, variationSelector)
```

7. What representation would I choose if I only cared about execution?

A sparse Unicode page table or comparable direct semantic map, plus a separate variation-selector structure.

Compiler interpretation

```
cmap formats
```

```
|
```

```
v
```

```
CharacterMap
```

```
|
```

```
v
```

```
code point -> glyph ID
```

13. Design Sheet - GSUB

`GSUB` is a feature-addressable glyph-sequence transformation program.

1. What mathematical/semantic function is implied?

```
GlyphSequence' = substitute(GlyphSequence, FeaturePlan)
```

```
applyLookup(LookupId, MutableGlyphBuffer)
```

2. What information does the font contribute?

Substitution rules, contextual patterns, script/language feature relationships, ordered lookups, lookup flags, glyph sets and glyph classes.

3. What algorithm does the engine contribute?

Matching, lookup filtering, glyph-buffer traversal and mutation, nested lookup execution, sequenceIndex behavior, recursion/operation guards, and rollback.

4. Which serialized distinctions exist mainly because of compression or storage?

Coverage and ClassDef format variants, Context and ChainContext formats 1/2/3, ExtensionSubst, raw offsets, and subtable boundaries.

5. Which distinctions have genuine semantic or performance meaning?

Single, multiple, alternate, ligature, contextual, chained-contextual and reverse-chain semantics; lookup/feature order; filtering; nested execution.

6. What would the API look like if OpenType did not exist?

```
featureLookups(FeatureId)
```

```
applyLookup(LookupId, GlyphBuffer)
```

7. What representation would I choose if I only cared about execution?

Stable lookup IDs, normalized lookup filters, GlyphSet/GlyphClassMap structures, semantic substitution payloads, common ContextRule and ChainContextRule forms.

Compiler interpretation

```
GSUB binary rule structures
```

```
|
```

```
v
```

```
semantic substitution program
```

```
|
```

```
v
```

```
small glyph-buffer executor
```

14. Design Sheet - GPOS

`GPOS` is a feature-addressable geometry and attachment program.

1. What mathematical/semantic function is implied?

```
position(GlyphBuffer, Geometry, FeaturePlan)
```

2. What information does the font contribute?

Single and pair adjustments, class-pair data, cursive entry/exit anchors, mark/base/ligature/mark anchors, contextual positioning rules, and variation-dependent values.

3. What algorithm does the engine contribute?

Lookup filtering, participant matching, geometry mutation, anchor resolution, mark attachment, cursive attachment, contextual matching, and nested lookup execution.

4. Which serialized distinctions exist mainly because of compression or storage?

Coverage/ClassDef variants, ExtensionPos, contextual formats 1/2/3, chained-contextual formats, raw offsets, subtable boundaries, and many anchor/value-record encoding details.

5. Which distinctions have genuine semantic or performance meaning?

Single and pair adjustment, cursive attachment, mark-to-base, mark-to-ligature, mark-to-mark, contextual positioning, lookup order, filtering, anchor class, ligature component, and variation semantics.

6. What would the API look like if OpenType did not exist?

```
adjust(glyph, adjustment)
adjustPair(left, right, leftAdj, rightAdj)
attachCursive(...)
attachMarkToBase(...)
attachMarkToLigature(...)
attachMarkToMark(...)
applyContextRule(...)
```

7. What representation would I choose if I only cared about execution?

ValueAdjustment, PairMap and/or PairClassMatrix, Anchor, attachment rules, common contextual rules, and variation values. The shaped glyph buffer can carry explicit attachment relationships.

Compiler interpretation

```
GPOS serialization
|
v
geometry + attachment semantics
|
v
small positioning executor
```

15. Design Sheet - GDEF

`GDEF` is a supporting glyph-semantics database used by layout execution.

1. What mathematical/semantic function is implied?

```
glyphClass(glyph)
markAttachmentClass(glyph)
markSetContains(set, glyph)
attachmentPoints(glyph)
ligatureCaret(glyph)
variationValue(index, coords)
```

2. What information does the font contribute?

Glyph roles (Base/Ligature/Mark/Component), mark attachment classes, mark filtering sets, attachment points, ligature carets, and variation-store information.

3. What algorithm does the engine contribute?

The executor decides whether glyphs should be ignored or included for a lookup and how class/set information affects lookup matching. Editing code can use ligature carets.

4. Which serialized distinctions exist mainly because of compression or storage?

ClassDef/Coverage formats, offset topology, mark-set serialization, and variation-store binary layout.

5. Which distinctions have genuine semantic or performance meaning?

Glyph role, mark attachment class, filtering-set membership, caret locations, and attachment points.

6. What would the API look like if OpenType did not exist?

```
GlyphRole role(glyph)
uint16_t markAttachmentClass(glyph)
bool belongsToMarkSet(setId, glyph)
Span<Caret> ligatureCaret(glyph)
```

7. What representation would I choose if I only cared about execution?

Compact per-glyph semantic properties plus shared glyph sets and sparse caret/attachment maps. Much of the table may be folded directly into compiled lookup filters.

Compiler interpretation

```
raw GDEF
|
+--> compiled lookup filters
+--> attachment metadata
+--> caret/editing metadata
+--> variation semantics
```

16. Design Sheet - BASE

`BASE` is structured relational data describing baseline geometry for scripts and languages.

1. What mathematical/semantic function is implied?

```
Coordinate baseline(script, language, baselineType, direction)
```

2. What information does the font contribute?

Baseline types and coordinates, per-script choices, optional language-specific data, horizontal/vertical baseline systems, extents, and variation-dependent coordinates.

3. What algorithm does the engine contribute?

Line layout decides which baseline is needed and computes how runs/fonts must be translated to align correctly.

4. Which serialized distinctions exist mainly because of compression or storage?

BaseCoord format variants, offset hierarchy, tag-list representation, script-record layout, and variation-reference encoding.

5. Which distinctions have genuine semantic or performance meaning?

Direction, script, language, baseline type, baseline coordinate, and extent data.

6. What would the API look like if OpenType did not exist?

```
baselineCoordinate(script, language, baselineKind, direction)
defaultBaseline(script, direction)
```

7. What representation would I choose if I only cared about execution?

A small script/language-to-baseline relational map or matrix.

Compiler interpretation

```
BASE serialized hierarchy
  |
  v
BaselineDatabase
  |
  v
script/language -> alignment geometry
```

17. Design Sheet - JSTF

`JSTF` is an ordered policy description for script/language-specific justification strategies.

1. What mathematical/semantic function is implied?

```
JustificationPlan justificationStrategies(script, language)
```

2. What information does the font contribute?

Prioritized suggestions for enabling/disabling GSUB and GPOS behavior and applying justification-specific positioning adjustments.

3. What algorithm does the engine contribute?

The layout engine computes the width delta, decides whether to expand or shrink, tries priority levels, measures the result, and integrates justification with whitespace and line breaking.

4. Which serialized distinctions exist mainly because of compression or storage?

Script/language hierarchy, raw lookup indices, offsets, and array organization.

5. Which distinctions have genuine semantic or performance meaning?

Priority order, shrink versus expand, enable versus disable, GSUB versus GPOS, adjustment limits, and script/language specificity.

6. What would the API look like if OpenType did not exist?

```
Span<JustificationStage> justificationPlan(script, language)
```

7. What representation would I choose if I only cared about execution?

An ordered `JustificationPlan` containing shrink/extend stages with stable semantic lookup IDs and normalized limits.

Compiler interpretation

```
JSTF
|
v
ordered justification policy
```

18. Design Sheet - hmtx

`hmtx` is a compact serialized database of nominal horizontal glyph metrics.

1. What mathematical/semantic function is implied?

```
HorizontalMetrics metrics(GlyphId glyph)
```

2. What information does the font contribute?

Per-glyph advance width and left side bearing.

3. What algorithm does the engine contribute?

Retrieve the metrics and initialize nominal geometry before GPOS adjustments.

4. Which serialized distinctions exist mainly because of compression or storage?

OpenType can omit repeated trailing advance widths and store only additional side bearings. This is a file-size optimization.

5. Which distinctions have genuine semantic or performance meaning?

The final advance width and left side bearing, including variable-font deltas where applicable.

6. What would the API look like if OpenType did not exist?

```
metrics(glyph)
```

7. What representation would I choose if I only cared about execution?

A direct HorizontalMetrics array or parallel advance/side-bearing arrays indexed by glyph ID.

Compiler interpretation

```
compressed hmtx
|
v
direct glyph metrics database
```

19. Design Sheet - vmtx

`vmtx` is the vertical analogue of `hmtx`.

1. What mathematical/semantic function is implied?

```
VerticalMetrics metrics(GlyphId glyph)
```

2. What information does the font contribute?

Per-glyph advance height and top side bearing.

3. What algorithm does the engine contribute?

Retrieve nominal vertical geometry; later GPOS stages may adjust positioning.

4. Which serialized distinctions exist mainly because of compression or storage?

Repeated advance heights can be compacted just as repeated horizontal advances are in hmtx.

5. Which distinctions have genuine semantic or performance meaning?

The actual advance height and top side bearing, including variation deltas.

6. What would the API look like if OpenType did not exist?

```
verticalMetrics(glyph)
```

7. What representation would I choose if I only cared about execution?

A direct VerticalMetrics array or parallel arrays indexed by glyph ID.

Compiler interpretation

```
compressed vmtx
```

```
|  
v
```

```
direct vertical metrics database
```

20. Compiled font faces and caching

Once OpenType is treated as a source language, the natural cache boundary becomes the compiled font face rather than the raw layout tables.

```
raw FontFace
|
+-- cmap
+-- outlines
+-- metadata
|
v
CompiledFontFace
+-- CharacterMap
+-- GlyphSemantics
+-- compiled GSUB
+-- compiled GPOS
+-- nominal metrics
+-- variation semantics
```

Compilation should be lazy. A font discovered during indexing may only need metadata and Unicode coverage. GSUB/GPOS compilation is necessary only when the face actually participates in shaping.

Because the IR uses stable IDs and semantic structures rather than pointers into the original font blob, it also opens a path toward serialization and memory mapping. A future cache could compile a font once and load the normalized representation directly on later runs.

A useful rule

Anything that can be decided while compiling the font should generally not be decided again while shaping text.

21. Testing becomes layered and much more surgical

The compiler boundary creates several independent test surfaces.

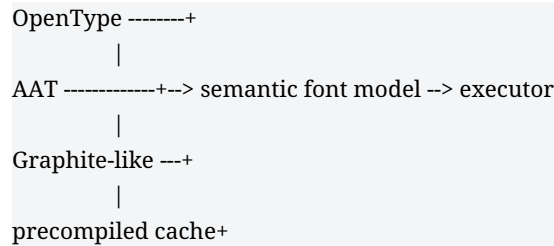
Layer	Test
OpenType -> IR compiler	Different binary encodings that mean the same thing should lower to equivalent semantic structures.
IR executor	Synthetic semantic rules can exercise matching, mutation, attachment, recursion, rollback, and edge cases without constructing a font file.
Full-font shaping	Font + text -> expected shaped glyph sequence and geometry.
Differential oracle	Compare SVG Buddy behavior against HarfBuzz across real fonts and scripts.

Synthetic IR tests are particularly valuable. A pathological contextual lookup graph can be expressed directly instead of hand-authoring Coverage tables, offsets, FeatureLists, LookupLists, and subtables merely to reach the behavior being tested.

This also splits fuzzing naturally: fuzz untrusted OpenType bytes into the compiler, and separately fuzz valid semantic IR into the executor.

22. What this enables beyond OpenType

A semantic runtime does not have to care where the semantics came from. OpenType is then one frontend rather than the definition of the execution engine.



There is no requirement to implement those alternate technologies. The architectural value is that doing so would not require replacing the text engine.

The same observation applies to rendering technologies. A glyph outline can come from glyf/loca or CFF/CFF2; a color glyph can come from COLR, SVG, or bitmap strikes. The renderer should consume realized geometry/paint semantics rather than mirroring every serialization detail.

23. What a user of the implementation should see

The public-facing API should remain substantially simpler than the internal machinery described in this paper.

At the application level, a caller should be able to provide:

- UTF-8 text
- style and font-selection policy
- font resources
- layout constraints
- rendering target

and receive a shaped/positioned glyph stream or directly rendered output.

```
text
-> Unicode pipeline
-> shaping runs
-> font runs
-> Script IR policy
-> compiled font semantics
-> ShapedGlyphBuffer
-> visual composition
-> absolute glyph positions
-> SVG / Blend2D rendering
```

The architectural work is valuable precisely because the application should not have to understand it. Complexity is concentrated behind stable semantic boundaries.

24. Practical implementation status

At the time of this design note, the GSUB migration is complete across all lookup types and has been differentially validated against the previous raw OpenType implementation. Context Formats 1/2/3 and ChainContext Formats 1/2/3 lower to common semantic representations; ExtensionSubst is compiled away.

The next major implementation step is GPOS, beginning with SinglePos and PairPos and then moving through cursive and mark attachment. That work should be driven by the same per-table questions rather than by a desire to mirror the specification's binary structure.

The Script IR is the next architectural layer after that. Thai is a useful first serious script because it exercises shaping-stream transformation, combining marks, mark positioning, OpenType feature staging, and source provenance while keeping paragraph direction LTR.

Arabic and Indic will then test whether the Script IR genuinely captures writing-system policy without leaking OpenType mechanics back into script-specific code.

25. Closing perspective

The important conceptual change is not a clever data structure. It is changing what the font specification is allowed to define inside the program.

The serialized OpenType format should define how information is transported and stored. It should not dictate the internal architecture of the shaping engine.

```
serialized format
|
v
validate + normalize + compile
|
v
semantic program state
|
v
small explicit APIs
```

Once that boundary is accepted, several things become clearer at once:

- Unicode character properties, script behavior, font capability, and rendering are different concerns.
- GSUB and GPOS are rule systems, not merely binary table layouts.
- GDEF is supporting glyph semantics rather than an independent shaping algorithm.
- Script policy belongs outside the font and can be represented explicitly.
- OpenType compression choices do not have to become runtime abstractions.
- Compiled font faces can be validated once, cached, serialized, inspected, and optimized.
- The shaping executor can become small enough to understand as an algorithm rather than as a tour through the OpenType specification.

The working mental model

Unicode describes characters. Script specifications describe writing systems. Font tables describe what a particular font can do. The two IRs make those responsibilities explicit, and the executor turns them into positioned glyphs.

Appendix A. Compact semantic map of the shaping-related tables

OpenType input	Semantic object	Best mental model
cmap	CharacterMap	Sparse function
GDEF	GlyphSemantics	Property/relationship database
GSUB	SubstitutionProgram	Declarative rule program
hmtx / vmtx	GlyphMetrics	Direct indexed database
GPOS	PositioningProgram	Rule + relationship program
BASE	BaselineDatabase	Relational data
JSTF	JustificationPlan	Policy program

Appendix B. Terms

Semantic IR: An internal representation that preserves the meaning needed by execution while discarding binary-format details that no longer matter.

Script IR: A small declarative program describing script-specific preparation, analysis, reordering, feature stages, metrics boundary, and positioning stages.

Font IR: The compiled semantic representation of font shaping capability: mappings, glyph semantics, substitutions, positioning rules, anchors, filters, and related data.

LookupId: A stable semantic identifier used instead of raw OpenType LookupList indices or pointers.

GlyphSet: A semantic set abstraction replacing direct runtime interpretation of Coverage table formats.

GlyphClassMap: A semantic glyph-to-class mapping replacing direct runtime interpretation of ClassDef formats.

ShapedGlyphBuffer: The glyph sequence plus geometry, cluster/provenance, and attachment state produced by shaping.

Source provenance: The relationship from transformed shaping items and produced glyphs back to the normalized Unicode source span that produced them.